Project Report: Deterministic Logic Engines in Game Theory

NAME :: ABHINAV GUPTA

REG NO. :: 25BAI10929

BRANCH :: CSE(AIML)

COURSE CODE :: CSA2001

COURSE TYPE :: FLIPPED COURSE

DATE :: MARCH 2026

Project: BYOP Challenge – Tic-Tac-Toe NumPy Model

1. Introduction

1.1 The Problem Statement

The challenge was to create a competitive AI for a zero-sum game (Tic-Tac-Toe) without relying on high-level machine learning libraries or recursive search algorithms (like Minimax).

1.2 Why It Matters

While Tic-Tac-Toe is mathematically "solved," implementing the solution using **primitive logic (if-else)** and **matrix manipulation** tests a developer's ability to map out every possible state-space. It demonstrates an understanding of how data structures (NumPy) interact with conditional decision-making.

---

2. Methodology & Approach

2.1 Technical Stack

- **NumPy:** Used for the 3×3 grid representation and vector-based win checking.
- **Conditional Logic:** A prioritized decision tree that simulates "intelligence."

2.2 The "Hard-Coded Tree" Strategy

Instead of the model "learning," I mapped the optimal move hierarchy:

1. **Immediate Win:** Scan rows, columns, and diagonals for a sum indicating two AI marks and one empty space.
2. **Immediate Block:** Scan for two Player marks and one empty space.
3. **Positional Advantage:** Prioritizing the center $(1, 1)$ or corners $(0, 0), (0, 2),$ etc.

---

3. Implementation Highlights

3.1 Matrix Evaluation

Explain how you used NumPy to check for wins efficiently. For example:

- np.sum(axis=0) for columns.
- np.trace() for the main diagonal.
- np.fliplr(board).trace() for the anti-diagonal.

3.2 Decision Branching

Describe the logic flow.

Example: "The model doesn't just look for an empty spot; it calculates the 'weight' of a move by passing the matrix through a series of conditional filters."

---

4. Key Decisions & Challenges

- **Decision:** Choosing a 3×3 NumPy array over a standard Python list.

    - *Reason:* NumPy allows for faster slicing and Boolean masking (e.g., board == 0).
- **Challenge:** Handling the "Fork."
    - *Solution:* If the player moves in two opposite corners, the AI must be hard-coded to take a side-edge rather than a corner to avoid a double-threat trap.
- **Challenge:** Matrix Indexing.
    - *Solution:* Translating human input (1–9) into matrix coordinates (r, c) using floor division and modulo operators.

---

5. Results & Analysis

- **Performance:** Does the model ever lose? (Ideally, a perfectly coded if-else Tic-Tac-Toe AI should never lose, only draw or win).
- **Efficiency:** Comparison of the "Tree Search" approach vs. a random move generator.

---

6. Lessons Learned

- **State-Space Complexity:** Even simple games have nuances that require rigorous logical mapping.
- **Vectorization:** How NumPy makes code cleaner compared to nested for-loops.
- **The "Brute Force" Realization:** Understanding that while if-else works for Tic-Tac-Toe, more complex games (like Chess) require the recursive algorithms I simulated here.

---

7. Conclusion

This project successfully demonstrates that a "smart" model can be constructed through foundational programming principles. By leveraging **NumPy** for data handling and **structured conditionals** for decision-making, I built an unbeatable opponent within the constraints of the BYOP challenge.

---

8. References & Appendix

- NumPy Documentation.
- Game Theory: Optimal Strategy for Tic-Tac-Toe.
- Source Code Link (https://github.com/2006abhi9-stack/TicTacToe)